

Contents

Arcade Library Extension Guide	1
Purpose	1
What The Core Expects	1
Implementing A New Graphics Library	1
Required Behavior	2
Recommended Library Contract	2
Implementing A New Game Library	2
Required Behavior	3
Practical Rules	3
Build And Packaging Rules	3
Minimal Export Template	3
Compatibility Checklist	3

Arcade Library Extension Guide

Purpose

Arcade is built around dynamically loaded shared objects. A new library must expose the same runtime contract as the existing ones so the core can discover it, load it, and use it through the common interfaces.

What The Core Expects

The core loads every `.so` from `./lib` and inspects it through three exported C symbols:

- `myEntryPoint()` returns a newly allocated instance of the module.
- `getLibType()` returns `arcade::EType::GRAPHICAL` or `arcade::EType::GAME`.
- `getLibName()` returns the human-readable name used in menus.

The loader is implemented in `src/DLLoader.hpp`. If one of these symbols is missing, the library is rejected.

Implementing A New Graphics Library

Create a class that inherits from `arcade::IDisplayModule` in `src/IDisplayModule.hpp`.

You must implement these public methods:

- `init(std::string name, Size size)`
- `getWindowSize() const`
- `stop()`
- `setBackground(Texture texture)`
- `createRect(Bounds bounds)`
- `setMusic(std::string filepath)`
- `playSound(std::string filepath)`

- `render()`
- `pollEvent()`
- `clearWindow() const`

You also need a concrete rectangle type that implements `arcade::IRect` from `src/IRect.hpp`:

- `getBounds() const`
- `isMouseOver() const`
- `setSize(Size size)`
- `setPosition(Position position)`
- `setText(std::string text, Texture texture)`
- `setBorder(Texture texture)`
- `setTexture(Texture texture)`
- `display() const`

Required Behavior

- `init` must create the rendering context and prepare the window for the requested size.
- `createRect` must return a shared rectangle object that can be moved, resized, styled, and displayed.
- `pollEvent` must translate the backend-specific input into `arcade::EEvent` values.
- `setBackground`, `setMusic`, and `playSound` should be no-ops only if the backend does not support them.
- `stop` must release every backend resource cleanly.

Recommended Library Contract

Keep the implementation stateless at the module boundary and let the core own the module lifetime. The current design creates the module through `myEntryPoint()` and destroys it when the dynamic library is closed.

Implementing A New Game Library

Create a class that inherits from `arcade::IGameModule` in `src/IGameModule.hpp`.

You must implement these public methods:

- `run(std::shared_ptr<IDisplayModule> display, const std::string name)`
- `pause()`
- `changeDisplay()`
- `changeGame()`
- `exit() const`

Required Behavior

- `run` receives the active display module and the player name.
- The game must draw everything only through the display interface and `IRect` instances.
- When the player requests a renderer switch, `changeDisplay()` must return `true` once, then reset its internal flag.
- When the player requests a game switch, `changeGame()` must return `true` once, then reset its internal flag.
- When the player requests to quit, `exit()` must return `true`.

Practical Rules

- Keep game state inside the game class.
- Never call backend-specific APIs from a game module.
- Use the key mapping defined by `arcade::EEvent` in `src/IDisplayModule.hpp`.

Build And Packaging Rules

The current Makefile builds the dynamic libraries into `./lib` with these expected names:

- `lib/arcade_sdl2.so`
- `lib/arcade_ncurses.so`
- `lib/arcade_gtk+.so`
- `lib/arcade_snake.so`
- `lib/arcade_pacman.so`

If you add a new module, keep the same `.so` convention so the launcher can discover it automatically.

Minimal Export Template

```
extern "C" arcade::YourDisplay *myEntryPoint();
extern "C" arcade::EType getLibType();
extern "C" std::string getLibName();
```

The implementation must return a display module for graphics libraries and a game module for game libraries.

Compatibility Checklist

- Inherit from the correct interface.
- Export the three required symbols.
- Return the correct `EType`.
- Use only the common `Texture`, `Size`, `Position`, `Bounds`, and `EEvent` types at the interface boundary.

- Make sure the module can be opened by `dlopen` and instantiated through `myEntryPoint()`.